

Team Project Report

Gaussian Blobs for 3D Fluorescence Microscopy

Fitting light-sheet volumes with 3D Gaussians,
and testing whether the nuclei survive

Syed Muhammad Asad (5301662)

Akim Al-Makhdar ([Matr.-Nr.])

Supervisor: Prof. Dr. Martin Weigert

October 10, 2026

Light-sheet microscopes record developing embryos as terabytes of 3D volumes. One proposed remedy is to store each volume as a set of 3D Gaussian blobs (“splats”), which is compact and renders quickly; such representations are judged almost only by PSNR. This project asked two questions. Can a Gaussian fit be made to keep the cell nuclei that biologists measure? And does the state-of-the-art Gaussian compressor keep them as well as a standard codec at the same file size? Starting from a synthetic phantom with a known answer, we tested changes one at a time on real *Tribolium* data: where blobs start, their shape, densification, a budget derived from the cell count, the position learning rate, and a size cap for nucleus blobs. Starting one blob on every detected nucleus raised the share of nuclei found after fitting from 52% to 71%, and the cap recovered 6 of 31 faint nuclei at no measurable PSNR cost; on 29 hand-labelled *Drosophila* nuclei, however, a classical blob filter put more of them into the top 100 detections than any fit. At identical bytes on 12 *C. elegans* frames with 2,958 labelled nuclei, JPEG2000 had higher PSNR than the Luxar Gaussian compressor on every frame, yet the number of nuclei found depended on the detector: below $300\times$ compression, 3D Cellpose found +3.5 percentage points more nuclei in splat reconstructions, while a watershed and 2D Cellpose found -4.0 and -9.4 points fewer. Beyond $300\times$, at fewer than about three splats per nucleus, splats lost 17–23 points for every detector. PSNR alone cannot decide which storage format is safe for a given analysis.

Contents

1	Introduction	4
1.1	The problem	4
1.2	What we set out to solve, and by how much	4
1.3	Scope: the project brief and what was delivered	4
1.4	Division of work	5
1.5	Outline	5
2	Related work	6
3	Data	7
4	Representation, fitting and evaluation	8
4.1	A volume as a sum of Gaussians	8
4.2	Fitting	8
4.3	Evaluation	8
5	From a simple setup to the final recipe	9
5.1	Step 0: a phantom with a known answer	9
5.2	Step 1 (Akim): start blobs on bright peaks	9
5.3	Step 2 (Akim): real data need stretched, rotated blobs	10
5.4	Step 3 (Akim): grow blobs where the error is large	10
5.5	Step 4 (Akim): set the budget from the cell count	11
5.6	Step 5 (ours): repeats confirm that stretching is real	11
5.7	Step 6 (ours): start one blob on every nucleus	11
5.8	Step 7 (ours): larger position steps let blobs travel	12
5.9	Step 8 (ours): cap the size of nucleus blobs	12
5.10	Why faint nuclei are lost	13
5.11	From the simple setup to the final recipe	13
5.12	Quality, file size and fitting time against the number of Gaussians	13
6	Validation: does the fit behave as expected?	14
7	Export and browser visualisation	16
8	Time-resolved extension and playback	17
9	Comparison with the state of the art at matched file size	19
9.1	Setup	19
9.2	Results	19
10	Discussion	22
11	Conclusion	22
	References	23
A	Glossary	25
B	Evaluation and pipeline bugs found	26
C	Pre-registered decision rules	26

D Reproducibility	27
-------------------	----

1 Introduction

1.1 The problem

Light-sheet microscopy images living embryos layer by layer, quickly and with little light damage, and records them as time series of 3D volumes. One frame of our *Tribolium* recording has 37 million voxels; one *C. elegans* recording of the Cell Tracking Challenge [1] has 195 frames and takes about 2.5 GB even at 8 bit, and real experiments run for hours and produce terabytes. Such data are hard to store, share and look at.

3D Gaussian splatting [2] offers a different way to store a volume: as a sum of a few thousand soft, stretched 3D blobs, each described by 11 numbers. The representation is continuous, compact and fast to draw on a graphics card. It has recently been adapted to microscopy [3, 4], to medical volumes for compact storage [5, 6] and to CT reconstruction [7, 8]. All of these are evaluated by global image similarity (PSNR, SSIM). For biologists the relevant question is different: after compression, does the analysis they run still find the cells? A nucleus is a small, often faint peak, and a smooth fit can lose it while the PSNR barely changes.

1.2 What we set out to solve, and by how much

The project addressed two concrete questions:

1. **Fitting.** Starting from the simplest working setup, which changes make a Gaussian fit keep the nuclei, and how much does each change help?
2. **State of the art.** At exactly the same number of bytes on disk, does the strongest existing Gaussian compressor (Luxar [3]) keep as many nuclei as a standard codec (JPEG2000), as judged by the detectors biologists use?

Table 1 summarises the answers. In short, our fitting changes made Gaussian fits keep nuclei much more reliably than the naive setup, but a classical blob filter applied to the raw image ranks nuclei better in the top 100 and as well in the top 200; and at matched file size the state-of-the-art splats are neither uniformly better nor uniformly worse than JPEG2000: the winner depends on the detector, which PSNR cannot reveal.

1.3 Scope: the project brief and what was delivered

The project brief set four goals: (1) fit Gaussians to a single microscopy volume and study quality against the number of Gaussians, file size, fitting time and GPU memory; (2) export the fit to .ply/.splat and inspect it in a browser; (3) extend the representation to time series, comparing per-frame Gaussian sets with shared identity against 4D Gaussians on storage, smoothness and frame-to-frame consistency; and (4) play the time series back in a browser. Table 2 maps each goal to the work in this report. Goal 1 was covered in depth and extended to object-level evaluation; Goals 2–4 were implemented in a basic form. In the second half of the project we added, on our own initiative, an object-level comparison of Gaussian compression with standard codecs (Section 9). It goes beyond the brief and is not one of its goals. Section 10 proposes compression that biologists use as a possible next step.

Question	Answer	By how much
Do more careful fits keep more nuclei? (our fitter, <i>Tribolium</i>)	Yes: start one blob per nucleus, cap the size of nucleus blobs	nuclei found 52% → 71%; faint nuclei 13 → 19 of 31; PSNR unchanged
Is the fit better than the raw image for finding labelled nuclei? (<i>Drosophila</i> , 29 labels)	Better than brightness ranking; worse than a blob filter in the top 100, level in the top 200	top-100 hits: fit 14, raw brightness 10, LoG filter 16–18
Do state-of-the-art splats keep more nuclei than JPEG2000 at the same bytes? (<i>C. elegans</i> , 12 frames)	Depends on the detector below 300 ×; never beyond	3D Cellpose +3.5, watershed −4.0, 2D Cellpose −9.4 points; beyond 300 × −16.6 to −23.2 points
Does PSNR predict this?	No: it prefers JPEG2000 everywhere	JPEG2000 higher on 12/12 frames and 80/80 pairs
Where do splats break? (<i>C. elegans</i> , LoG)	Below about three splats per nucleus	nuclei kept crosses 90% at 1.1–3.1 splats per nucleus, on 12/12 frames

Table 1: Summary of outcomes. “Points” are percentage points of *nuclei kept* (nuclei found after compression divided by nuclei found in the raw volume by the same detector); negative means JPEG2000 kept more. Details in Sections 5, 6 and 9.

1.4 Division of work

The project was carried out by two students. **Akim Al-Makhdar** built the first fitting pipeline and ran the exploratory experiment archive (399 fits in 57 experiments): the phantom experiments on starting positions, the sweep over blob shapes and budgets on real data, densification, and the cell-count budget (Steps 1–4 in Section 5). **Syed Muhammad Asad** re-analysed this archive and built the evaluation framework (physical units, object-level matching, regression tests), ran the controlled experiments on repeats, seeding, learning rate and size cap (Steps 5–8), the check against human labels (Section 6), and the state-of-the-art comparison (Section 9). The phantom (Step 0) and the representation were shared work.

1.5 Outline

Section 2 places the work in context, Section 3 describes the data and Section 4 the representation, fitting and evaluation. Section 5 goes from a phantom to the final fitting recipe, one change at a time, stating for each change what was changed, why it should help, and what it did. Section 6 checks whether the fits behave as expected and compares them with naive baselines. Section 7 covers export and the browser viewer, Section 8 the time-resolved extension and playback, and Section 9 compares Gaussian compression with standard codecs at matched file size. Section 10 discusses limitations and the next steps. A glossary of technical terms is in Appendix A.

Goal of the brief	What was done	Where
1. Static fitting: quality vs. number of Gaussians, file size, fit time, GPU memory	Done and extended: fitting experiments, object-level evaluation, human labels. Our fitter is plain PyTorch with voxel queries, not gsplat: its render-and-compare objective was biased for volumes (Step 0). GPU memory was not measured.	Sections 4 to 6
2. Export to .ply/.splat, browser inspection	Done: exporter with exact geometry round trip; own browser viewer. Opening the files in SuperSplat or antimatter15/splat was not tested for this report.	Section 7
3. Time series: shared identity vs. 4D Gaussians	Partial: three representations implemented and compared on a synthetic phantom at matched storage, with held-out frames; no usable real-data comparison.	Section 8
4. Time-series renderer	Partial: the viewer steps through per-frame sets with a slider (discrete frames); the 4D model’s continuous time is not played back in the browser.	Section 8
Added by us, beyond the brief	Object-level comparison of Gaussian compression with standard codecs at matched size.	Section 9

Table 2: The four goals of the project brief and their status.

2 Related work

Gaussians for nuclei. Gaussian mixtures have long been used to segment and track nuclei in light-sheet data, most prominently TGMM [9], which also limits the covariance of each Gaussian. Representing nuclei as Gaussians, or capping their size, is therefore not new on its own; our question concerns Gaussians as a *storage format*.

Gaussian volume representations. 3D Gaussian splatting [2] renders opaque scenes for a camera by alpha-compositing; dynamic scenes are modelled by Gaussians that move with a learned deformation field [10]. It has been adapted to tomographic reconstruction [7, 8], to slice-to-volume MRI with closed-form point-spread functions [11], and to the compression of microscopy and medical volumes [3, 4, 5, 6]. Luxar fits splats to large microscopy volumes in minutes on one GPU and chooses the number of splats by a self-supervised held-out PSNR [12]; its authors describe the fits as intended for visualisation rather than quantitative analysis. All of these works report global image fidelity.

Compression of microscopy data. Biologists mostly store raw data losslessly. B3D compresses light-sheet data in real time, losslessly or with errors bounded by the camera noise [13], and the adaptive particle representation stores a volume as particles whose density follows the local image resolution [14]. Lossy image codecs such as JPEG2000 [15] and JPEG-XL [16] are the natural baselines at matched size. In radiology, lossy compression is judged by whether lesions remain detectable, not by PSNR [17]; Gaussian volume compression has not yet been held to that standard.

Detection and labels. We use Cell Tracking Challenge annotations [18, 1], a scale-normalised Laplacian of Gaussian (LoG) [19], a distance-transform watershed, and Cellpose [20].

Dataset	Voxel size $z \times y \times x$ (μm)	Reference nuclei	Use
Synthetic phantom	isotropic	exact (generated)	Step 0: does the fitting code recover a known answer?
<i>Tribolium castaneum</i> (frame i000022)	$3.0 \times 0.69 \times 0.69$	detector-derived	Development: $64 \times 128 \times 128$ crops and the whole volume ($71 \times 1024 \times 512$)
<i>Drosophila</i> , Fluo-N3DL-DRO (CTC)	$2.03 \times 0.406 \times 0.406$	189 human-labelled tracks	Validation against human labels; a selected subset is labelled, so precision is not interpretable
<i>C. elegans</i> , Fluo-N3DH-CE (CTC)	$1.0 \times 0.09 \times 0.09$	every nucleus labelled	State-of-the-art comparison: 12 frames of two embryos, 2,958 nuclei

Table 3: Datasets. CTC: Cell Tracking Challenge [1]. Detector-derived references are positions found by a peak detector; human labels are positions marked by experts and are the only non-circular reference.

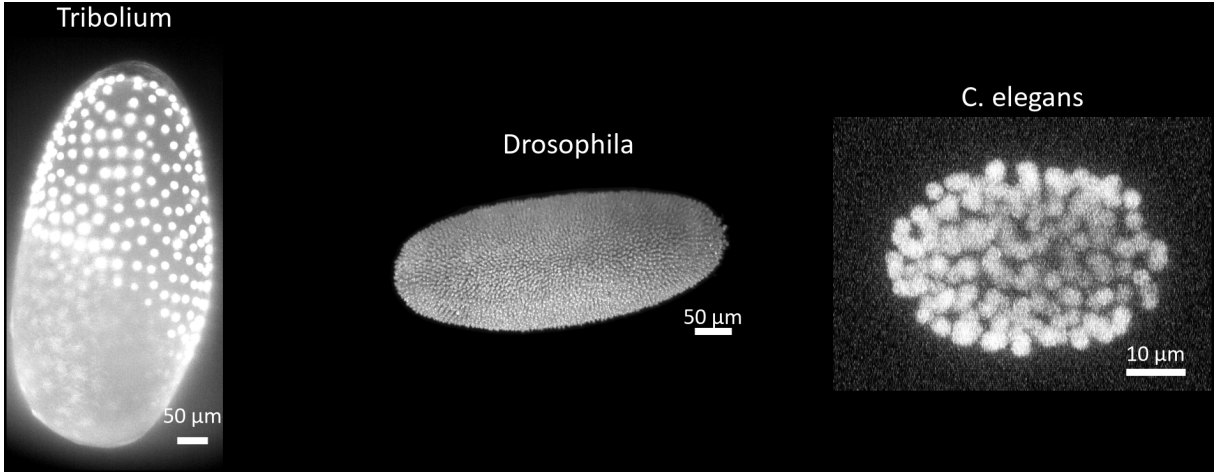


Figure 1: The three real recordings, shown as maximum-intensity projections (for each line of sight the brightest voxel). Nuclei are fluorescently labelled and appear as bright blobs.

3 Data

We used three real light-sheet recordings of developing embryos and one synthetic phantom (Table 3 and Figure 1). The depth step is much larger than the pixel size, so nuclei look stretched along z ; every spatial setting in this work is therefore given in micrometres and converted per axis, never in voxels.

For the state-of-the-art comparison we used six frames of each of the two *C. elegans* embryos (sequence 01: t110, t130, t150, t170, t185, t194; sequence 02: t110, t130, t150, t165, t180, t185), with 102–362 labelled nuclei per frame. Four frames were chosen first; eight were added later under rules fixed before the data were analysed (Appendix C). Embryo 1 t100, whose early nuclei are larger, is reported separately. Nuclei measure about $D = 3.6 \mu\text{m}$, and all spatial parameters of that study are fixed multiples of D .

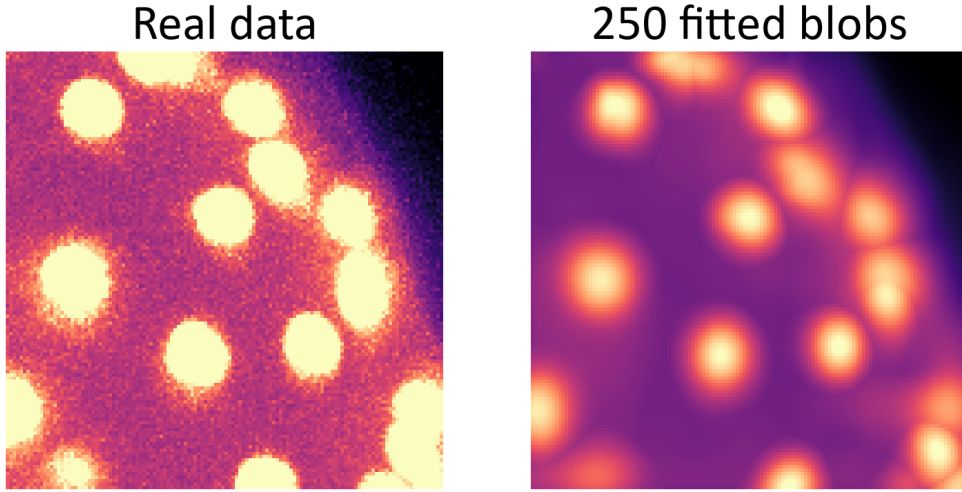


Figure 2: A $64 \times 128 \times 128$ *Tribolium* crop (left) and its reconstruction from 250 fitted Gaussians (right), maximum projections along z , each scaled to its own 1st–99.5th percentile. Nuclei are kept as smooth blobs; the fine texture is smoothed away.

4 Representation, fitting and evaluation

4.1 A volume as a sum of Gaussians

A volume $I(\mathbf{x})$ is approximated by K Gaussian blobs,

$$\hat{I}(\mathbf{x}) = \sum_{k=1}^K a_k \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right), \quad \Sigma_k = R(\mathbf{q}_k) S_k S_k^\top R(\mathbf{q}_k)^\top, \quad (1)$$

with position $\boldsymbol{\mu}_k \in \mathbb{R}^3$, scales $S_k = \text{diag}(s_{k,x}, s_{k,y}, s_{k,z})$ stored as logarithms, rotation R given by a unit quaternion $\mathbf{q}_k$, and amplitude a_k . Each blob stores 11 numbers ($3 + 3 + 4 + 1$) and has 10 degrees of freedom, because the quaternion is normalised. Simpler variants fix the rotation (*axis-aligned* or *stretched*), tie the three scales (*isotropic* or *round*), or fix the scale altogether. Figure 2 shows a real crop and its fit with 250 blobs: 2,750 stored numbers instead of about one million voxels. This is a count of numbers, not a measured file size; file sizes are compared in Section 9.

4.2 Fitting

The fit evaluates Equation (1) directly at voxel centres and minimises the squared error to the data with Adam [21], typically for 3,000 iterations. Each iteration uses a random batch of voxels: 70% drawn in proportion to intensity and 30% uniformly, without importance correction, so the effective objective weights bright voxels more than plain squared error. Ten percent of voxels are held out for validation. Held-out PSNR measures interpolation within a volume, not generalisation, because the initialisation sees the whole volume. We do *not* use the alpha-compositing renderer of 3D Gaussian splatting, for the reason given in Step 0.

4.3 Evaluation

Image quality. The peak signal-to-noise ratio (PSNR, in dB) between reconstruction and data; higher is better, and +3 dB roughly halves the squared error.

Object fidelity. A detector is run on the reconstruction and its detections are matched one-to-one to reference nuclei within a radius. Three rules turned out to be essential, each learned from a bug (Appendix B):

- **Physical units.** Detection windows, smoothing and matching radii are set in micrometres. A 13-voxel cubic window spans $26\mu\text{m}$ in z but $5.3\mu\text{m}$ in xy on *Drosophila*; with it the detector recovered 5 of 29 labelled nuclei from the raw volume, with a physical window 22 of 29.
- **Maximum-cardinality matching.** Minimum-cost assignment followed by a radius filter can drop valid matches; out-of-radius pairs are now penalised so that the number of matches is maximised first.
- **Matched budgets.** Recall grows with the number of detections, so methods are compared at the same number of detections (top N).

We report *recall* (share of reference nuclei found), *F1* (harmonic mean of recall and precision) and, for compression, *nuclei kept*: nuclei found after compression divided by nuclei found in the raw volume by the same detector. A value of 1 means that the same number of labelled nuclei was found, not necessarily the same individual nuclei. A suite of 24 regression tests pins every evaluation bug we found.

5 From a simple setup to the final recipe

We started from the simplest setup that can be checked exactly, a synthetic phantom, and then tested one change at a time on real data, each in its own comparison under the settings stated there. For each change we state what was changed, why it was expected to help, and what it did. Steps 1–4 are Akim’s experiments, Steps 5–8 ours.

5.1 Step 0: a phantom with a known answer

Setup. A $64 \times 128 \times 128$ volume made of 50 Gaussian blobs of random size, orientation and brightness, fitted with 2,000 Gaussians for 3,000 iterations by comparing the Gaussian sum with the volume at every voxel. **Why.** The phantom is itself a Gaussian sum, so correct code must recover it almost exactly. **Result.** The fit reached 47.9 dB (Figure 3).

We also learned here that the renderer of 3D Gaussian splatting is the wrong model for fluorescence volumes. It alpha-composites: each blob partly hides the blobs behind it, which is correct for opaque surfaces seen by a camera but wrong for transparent, light-emitting tissue. On a 15-blob phantom, the same fit scored 31.7 dB when compared at voxels but -7.1 dB through the alpha-compositing renderer. All later fits therefore compare blobs and voxels directly.

5.2 Step 1 (Akim): start blobs on bright peaks

Change. Random starting positions were replaced by one blob on each bright local maximum of the data. **Why.** A blob that starts far from any structure contributes nothing and may never reach one. **Result.** On the phantom (best of 8 fits per setting: four blob shapes, 50 and 100 blobs), random starts reached 32.0 dB and more iterations added only 1.7 dB (33.7 dB); starting on peaks reached 53.1 dB, and 55.1 dB with more iterations. Where blobs start mattered far more than how long they were fitted. Step 7 later explains why.

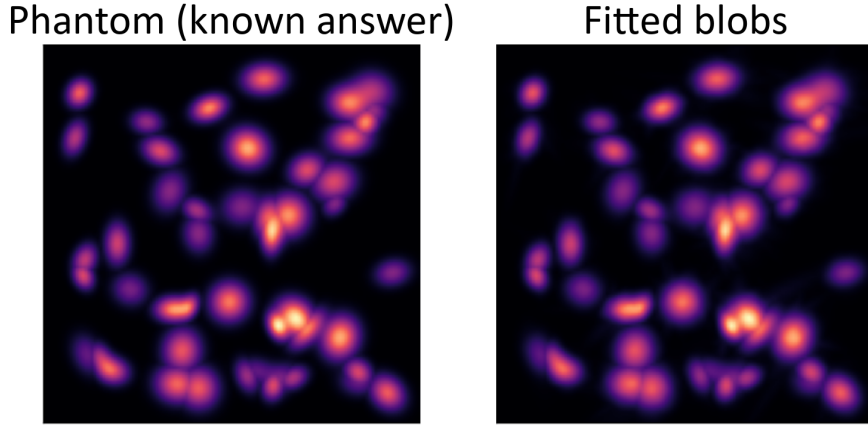


Figure 3: Step 0. Synthetic phantom (left) and its fit with 2,000 Gaussians (right), maximum projections; PSNR 47.9 dB.

Blob shape	Median PSNR (dB), Step 2	Mean PSNR (dB), Step 5
Fixed round (size not learned)	6.8	—
Round, learned size	20.4	22.3
Stretched (axis-aligned)	23.3	25.2
Stretched and rotated	24.0	25.5

Table 4: Blob shape on real *Tribolium* data. Step 2: Akim’s archive, about 75 fits per shape with different budgets and settings, one run each. Step 5: our repeat with 5 seeds and 3 budgets per shape (15 fits each) on $64 \times 128 \times 128$ crops.

5.3 Step 2 (Akim): real data need stretched, rotated blobs

Change. Moving to real *Tribolium* data (43 experiments, 299 fits), Akim compared four blob shapes and varied the budget (50–1,000 blobs), the voxel sampling, the loss, and an automatic tuner that estimates background, noise, nucleus size and spacing from the data. **Why.** Real nuclei, unlike the phantom, are stretched along z , unevenly bright and tightly packed. **Result.** The median PSNR rose steeply with shape freedom (Table 4). The best fit in the archive reached 25.9 dB with 750–1,000 blobs on the whole 37-million-voxel volume; this is the best observed value, not a proven limit. Fixed round blobs failed largely because their size was fixed at 2 voxels while nuclei need 5–7, so part of that gap is the constant, not roundness itself.

5.4 Step 3 (Akim): grow blobs where the error is large

Change. Densification, a standard trick of 3D Gaussian splatting: fitting starts with 25 blobs and splits or clones blobs where the error stays high. Akim tried 12 variants (when to add blobs, whether new blobs inherit their parent’s size and brightness, how many voxels to sample). **Why.** The data, rather than a guess, would decide how many blobs are needed. **Result.** At similar final numbers of blobs, grown fits were slightly better than fixed budgets, for example 24.3 dB with 34 grown blobs against 23.8 dB with 50 fixed blobs (Figure 4a). The best grown fits (24.9 dB) used about 165 blobs and 5,000 iterations, against 24.5 dB for the largest fixed budget (100 blobs, 3,000 iterations). The gain is modest, and we kept fixed budgets because they are easier to control and compare.

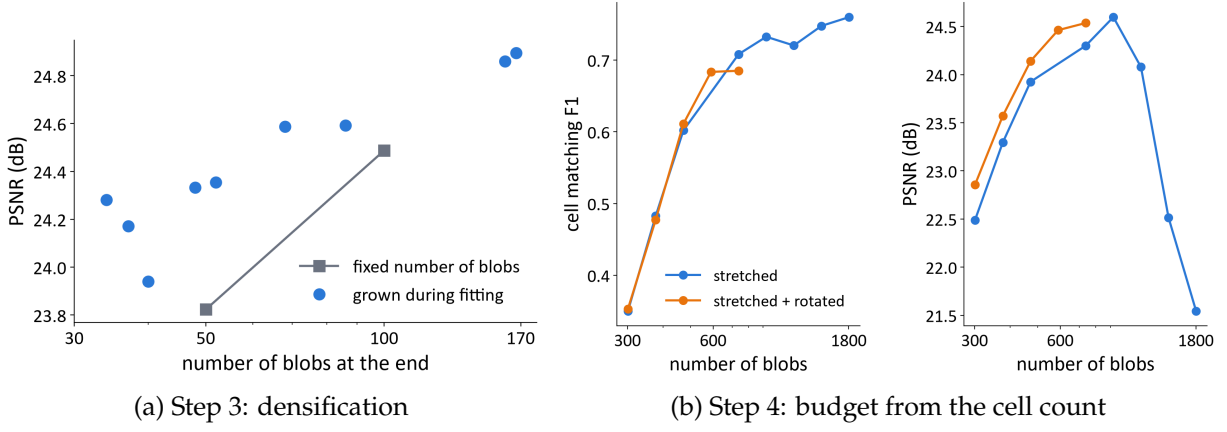


Figure 4: Akim’s experiments on *Tribolium*. (a) PSNR against the final number of blobs, same crop, stretched and rotated blobs: fits grown during fitting (dots) and fixed budgets (squares). (b) Whole volume, budgets set from 230 detected cells: agreement between peaks of the reconstruction and the detected cells (left, F1) and PSNR (right). All runs stopped after 3,000 iterations, which under-trains the largest budgets.

5.5 Step 4 (Akim): set the budget from the cell count

Change. Instead of guessing K , a detector first counted about 230 cells in the whole volume; the budget was then 1.15 blobs per cell plus 12% for background (301 blobs), and multiples of it. **Why.** If every cell gets a blob, the blobs should match the cells. **Result.** Agreement between peaks of the reconstruction and the detected cells rose steadily with the budget, from F1 0.35 at 301 blobs to 0.76 at 1,804 (Figure 4b). PSNR, however, peaked near 900 blobs and then fell, because every run stopped after 3,000 iterations. This was the first sign that image quality and finding cells do not always agree. The F1 gain comes with a sixfold larger budget: it shows that more blobs match more cells, not that deriving the budget from the cell count is better than another way of choosing it, which was not compared. The reference cells here come from a detector, not from people.

5.6 Step 5 (ours): repeats confirm that stretching is real

Change. We re-ran round, stretched, and stretched-and-rotated blobs with 5 random seeds and 3 budgets each. **Why.** Akim’s archive had one run per setting, and a single run can be luck. **Result.** Stretching held up: 22.3 dB (round), 25.2 dB (stretched), 25.5 dB (stretched and rotated), averaged over budgets of 50, 100 and 250 blobs (Table 4). The ordering held at every budget, and the standard deviation over seeds was 0.02–0.13 dB within each shape and budget, far below the 3 dB effect. Most of the gain comes from stretching; rotation adds about 0.3 dB. Nucleus F1 against detector-derived references rose from 0.745 to 0.805 (paired test over 5 seeds, $p = 0.006$). The learned blobs were also about as stretched as the real nuclei: depth-to-width ratio 0.79 against 0.73 measured on the nuclei.

5.7 Step 6 (ours): start one blob on every nucleus

Change. Peak starts (Step 1) were replaced by a start that first places one blob on every detected nucleus and then fills the remaining budget. **Why.** We measured that blobs travelled less than one voxel on average during fitting, while nuclei are about 20 voxels apart, so a blob that starts between nuclei stays there. With brightest-peak starts, bright background crowded out faint

Position learning rate	Mean travel (voxels)	Moved to another nucleus	F1 gap (good – random starts)	F1, good starts
0.0016 (default, 1×)	0.96	1.0%	+0.207	0.842
0.005 (3×)	2.47	5.5%	+0.169	—
0.016 (10×)	5.81	14.5%	+0.155	—
0.05 (31×)	10.76	28.0%	+0.048	—
0.1 (62×)	14.79	32.0%	+0.001	0.776

Table 5: Step 7: position learning rate (*Tribolium*, 100 blobs, 3,000 iterations, 2 seeds). A blob *moved to another nucleus* if its nearest nucleus changed during fitting. F1 against detector-derived references.

	Region 1	Region 2	Region 3	Region 4	Pooled
No cap	4/9	5/13	3/4	1/5	13/31
Cap 10 voxels	5/9	9/13	2/4	3/5	19/31

Table 6: Step 8: faint nuclei still detected after fitting, four *Tribolium* regions, seed 0, default position learning rate. Overall recall (mean over regions) 0.730 $\rightarrow$ 0.800; global PSNR 24.758 $\rightarrow$ 24.746 dB.

nuclei. **Result.** Over four *Tribolium* regions (110 nuclei) the share of nuclei with a blob at the start rose from 73% to 100%, and the share found after fitting from 52% to 71%. These references come from a related detector, so this result is partly circular; on human-labelled *Drosophila* the specific window rule did not transfer, whereas setting all windows in micrometres did.

5.8 Step 7 (ours): larger position steps let blobs travel

Change. We raised the learning rate of blob positions by up to 62×. **Why.** The default had been copied from photographic 3D splatting code, where coordinates are small, and was never rescaled to voxel coordinates. **Result.** Blobs started to move between nuclei (Table 5). With bad starts this closed the gap completely (F1 0.64 $\rightarrow$ 0.78), but with good starts it hurt (0.84 $\rightarrow$ 0.78). The position optimiser had been under-scaled rather than trapped in local minima, which is why starting positions mattered so much in Steps 1 and 6. Good starts with the default step remained best.

5.9 Step 8 (ours): cap the size of nucleus blobs

Change. An upper limit of 10 voxels on the scale of the blobs that started on nuclei (6.9 μm in xy but 30 μm in z , so in effect a cap in xy). **Why.** The nuclei still lost were the faint ones. At the end of fitting, the blob nearest to each lost nucleus was not dimmer than for kept nuclei (amplitude $p = 0.63$) but wider (scale 11.7 against 9.1 voxels, $p = 0.001$), and a wide blob leaves only a flat peak for a detector to find. A cap on width should therefore help. This is an association at the end of fitting, not a tracked trajectory of individual blobs, and it was scored before the matcher fix (Appendix B) and could not be rescored. **Result.** Faint nuclei found rose from 13 to 19 of 31 and overall recall from 0.730 to 0.800, while global PSNR changed by -0.012 dB (Table 6 and Figure 5). The result rests on one seed and four regions, one region got worse, and on human-labelled *Drosophila* the cap changed nothing because it bound only 2% of blobs there. It is promising, not settled.

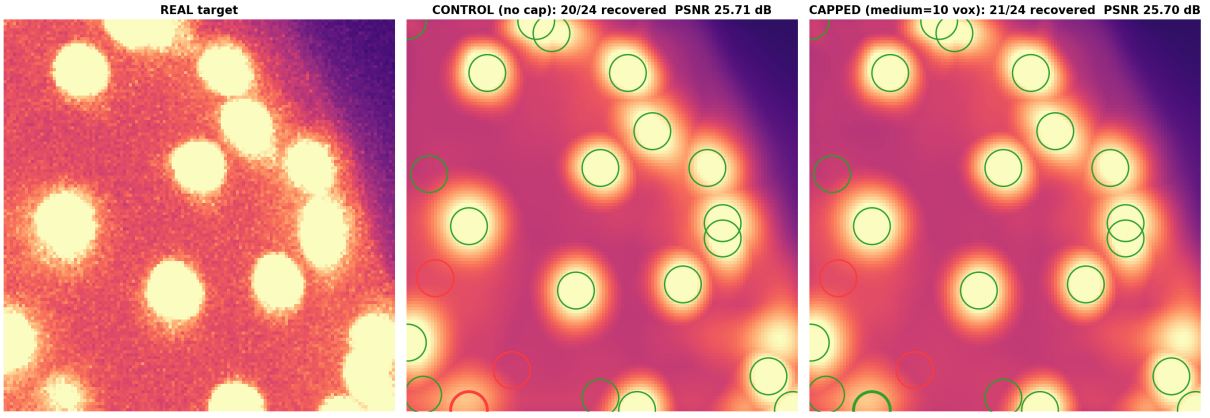


Figure 5: Step 8 on one *Tribolium* crop with 24 detector-found nuclei. Left: data (maximum projection). Middle: fit without cap, 20 of 24 nuclei recovered. Right: capped fit, 21 of 24. PSNR 25.71 against 25.70 dB.

5.10 Why faint nuclei are lost

Two measurements explain much of what the steps above fought against. First, most of the blob budget models background: in two regions with 250 blobs each, 62 of 63 nuclei were covered by exactly one blob, but only 31–33 blobs sat on a nucleus, carrying 10–17% of the total amplitude; the remaining 87% of blobs model a near-constant background. Under squared error a missed faint nucleus costs far less than a small error spread over the background, so the objective rationally spends capacity there. A learnable background term did not help, because with 250 broad blobs the split between the term and the blobs is not identifiable. Second, the blobs associated with lost faint nuclei at the end of fitting were wider than those of kept nuclei, and capping their width improved retention (Step 8). This supports broadening as a contributor to losing faint nuclei under this fitting regime. We do not claim a proven causal chain: the comparison uses the blob nearest to each nucleus at the end of fitting, not blobs tracked through fitting.

5.11 From the simple setup to the final recipe

Table 7 lists all eight changes. They are separate comparisons under the settings stated in each step, not a chain in which each row builds on the one before. They used different data, budgets and numbers of repeats, so they cannot be added up into one overall improvement, and row 4 compares budgets, not ways of choosing a budget. The final recipe is: stretched and rotated blobs, one starting blob per detected nucleus, the default position learning rate with these good starts, and a size cap for nucleus blobs. A combination of a larger learning rate with the cap was never run and is not part of the recipe.

5.12 Quality, file size and fitting time against the number of Gaussians

Goal 1 of the brief asks how quality trades against the number of Gaussians, file size, fitting time and GPU memory. Table 8 gives one sweep on a $64 \times 128 \times 128$ *Tribolium* crop (stretched and rotated blobs, 1,000 iterations, laptop CPU). Held-out PSNR barely moved: 0.6 dB from 100 to 1,000 Gaussians, and slightly lower at 2,000, while fitting time grew in proportion to the number of Gaussians, because every sampled voxel is evaluated against every Gaussian. The fixed 1,000 iterations probably under-train the largest budgets, as in Step 4. File size is fixed per Gaussian: 32 bytes in .splat and 68 in .ply (Section 7). GPU memory was not measured: our fits ran on

6 Validation: does the fit behave as expected?

Step	Change	By	Measured as	Before	After
1	Start on bright peaks	Akim	PSNR, phantom (best of 8)	32.0 dB	53.1 dB
2	Stretched + rotated blobs	Akim	PSNR, real data, median	20.4 dB	24.0 dB
3	Grow blobs during fitting	Akim	PSNR, 34 grown vs 50 fixed	23.8 dB	24.3 dB
4	Budget in cell-count units	Akim	cell-matching F1, 301 $\rightarrow$ 1,804 blobs	0.35	0.76
5	Repeat of the shape test	ours	PSNR, 15 fits per shape	22.3 dB	25.5 dB
6	One start per nucleus	ours	nuclei found, of 110	52%	71%
7	Larger position steps	ours	F1, random starts	0.64	0.78
8	Size cap	ours	faint nuclei found, of 31	13	19

Table 7: The eight changes from the naive setup (round blobs, random starts, default steps, no cap): separate comparisons under the settings stated in each step. The rows are not cumulative. Row 4 shows F1 as the budget grows sixfold; it does not compare the cell-count rule with another way of choosing the budget.

Gaussians	Held-out PSNR (dB)	Stored numbers	.splat size (kB)	Fit time (min)
100	22.5	1,000	3.2	1.0
250	22.7	2,500	8.0	2.7
500	22.9	5,000	16.0	5.3
1,000	23.1	10,000	32.0	11.9
2,000	22.9	20,000	64.0	22.3

Table 8: Capacity sweep on one *Tribolium* crop (runs/capacity_tribolium); the raw crop is 2 MiB at 16 bit. Stored numbers count 10 degrees of freedom per Gaussian.

a laptop CPU, and the GPU fits of Section 9 used Luxar, whose fitting took 17–46 min per whole *C. elegans* frame on an A100.

6 Validation: does the fit behave as expected?

Shape. The learned blobs are about as stretched as the real nuclei (depth-to-width ratio 0.79 against 0.73), as expected if they describe nuclei rather than arbitrary texture.

Human labels. The *Tribolium* references above come from a detector, so a fit can look good by agreeing with it. We therefore tested on 29 human-labelled *Drosophila* nuclei in one $64 \times 128 \times 128$ crop. All fits started from the same 457 detector candidates of the raw volume, so only the fitting differed, and methods were compared at the same number of detections (Table 9 and Figure 6). Fitting ranked true nuclei above image brightness in every seed (14 against 10 of 29 in the top 100). But a scale-normalised LoG filter applied to the same raw candidates did better in the top 100 (16–18), in seconds; in the top 200 the best fit and the filter were level (22.7 against 22, within one nucleus). The raw detector using all 457 candidates found 26 of 29, more than any fit. The fit’s advantage over brightness is explained by blob-shaped scoring, not by anything specific to the Gaussian representation. A large position learning rate hurt good starts here too, as Table 5 predicts, and the best-PSNR fit found fewer nuclei in the top 100 than the default fit: image quality and object fidelity come apart. With 29 labels one nucleus is 3.4 recall points, so we call a difference real only if it is about three nuclei or more and has the same sign in every seed.

Repeatability. Repeated fits differ little: in Step 5 the spread between seeds was small against the 3 dB shape effect, and three independent Luxar fits of the same frame (Section 9) differed by at most 2.1–3.7 points of nuclei kept.

Method	Top 100	Top 200	Notes
Raw volume, ranked by brightness	10	18	26/29 using all 457 candidates
Downsampled to similar storage	10	11	17.9 dB vs. 21.5 dB for the fit
Gaussian fit, default	14.0	21.0	best single fit reaches 23
+ size cap (17 voxels)	14.0	21.0	cap binds 2% of blobs
+ uniform sampling	11.7	22.7	best PSNR of all fits (+0.4 dB)
+ large position learning rate	11.0	18.3	
Raw candidates, LoG multi-scale (1.5–3 μm)	16	22	seconds, deterministic
Raw candidates, LoG 2 μm	18	22	scale chosen after seeing results

Table 9: Human-labelled *Drosophila* nuclei (of 29) among the top- N detections. Fit rows are means of 3 seeds. Paired per-seed differences, fit minus raw brightness: +3, +5, +4 (top 100) and +2, +4, +3 (top 200).

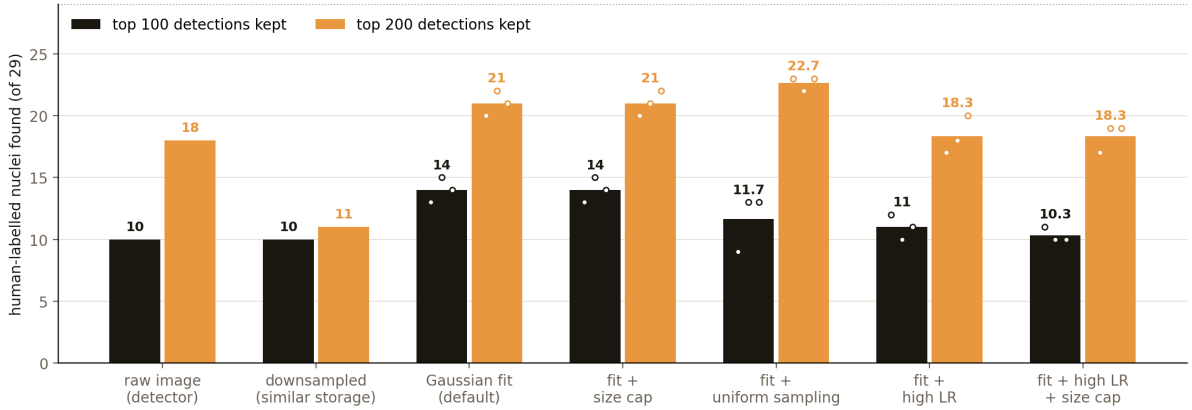


Figure 6: Human-labelled nuclei found in the top 100 and top 200 detections per method (bars: mean; dots: seeds). The LoG baseline of Table 9 is not drawn: it exceeds every fit in the top 100 (16–18), and in the top 200 (22) it is level with the uniform-sampling fit (22.7).

Naive baselines for other uses. We also tested whether the fitted representation helps beyond storage. Every use lost to a simple baseline (Table 10), which motivated the comparison with the state of the art in the next section.

Use	Outcome
Compression	Blobs lead at very small budgets (1,000 numbers: 23.4 vs. 21.6 dB), but plain downsampling catches up by about 10,000 numbers (23.1 vs. 23.3 dB). Going from 100 to 2,000 blobs bought only 0.36 dB.
Detection	The raw detector with all candidates finds more; LoG ranks better (Table 9).
Tracking by persistence	A plain detect-then-link tracker already keeps 76–83% of the 189 annotated tracks over 20 frames; linking real detections is 96.2% correct.
Axial super-resolution	Linear and cubic interpolation were as good (not run head-to-head on identical held-out slices).
Time series	Sharing blob identities over time used $3.4\times$ more parameters (13,200 vs. 3,900), not fewer.
Cell divisions	Untestable: none of the annotated tracks divide.

Table 10: Other uses of the representation that were tested and lost to a simple baseline.

7 Export and browser visualisation

Formats. A fitted set of Gaussians is written in two standard formats (`volspat/export.py`, `scripts/export_splat.py`): the PLY layout of 3D Gaussian splatting (17 floats, 68 bytes per Gaussian), read by SuperSplat and similar tools, and the compact `.splat` layout of `antimatter15/splat` (32 bytes per Gaussian: position and scales as floats, colour and opacity as 8-bit values, rotation as four 8-bit values). Microscopy has no colour, so the fitted brightness of each Gaussian is mapped to a grey value and an opacity.

What survives the export. Re-reading the PLY returned positions and scales exactly (maximum error 0 on the *Tribolium* and *Drosophila* fits below). The `.splat` format stores rotations in 8 bits per component, which changed orientations by at most 1.6° (median 0.9°). Brightness is a display mapping, normalised to the 99th percentile, clipped and stored in 8 bits, so the exported files are for viewing, not for quantitative analysis. Sizes: the *Drosophila* fit of Section 6 (500 Gaussians) takes 16.0 kB as `.splat` and 34.4 kB as `.ply`, against 2.1 MB for its 16-bit crop; a 12,000-Gaussian fit of a $64 \times 256 \times 256$ *Tribolium* crop takes 384 kB and 816 kB, against 8.4 MB.

Viewer. We inspect the files in our own browser viewer (`viewer/index.html`, `three.js`), shown in Figure 7. It loads one or several `.splat` files by drag and drop or from the page address, and supports orbiting, zooming, a point-size control and a threshold that hides faint Gaussians, which mostly model background (Section 5.10). It draws each Gaussian as a round, soft sprite sized by its average scale; the orientation and anisotropy of the Gaussians are not drawn, so it is a point-sprite viewer rather than a full splat rasteriser. Anisotropic voxels are corrected for display by stretching z (e.g. $5\times$ for *Drosophila*). For this report we added loading from the page address, the z stretch, the threshold and a cap on sprite size, and fixed a bug that centred every time point separately (Section 8). Opening the same files in SuperSplat or `antimatter15/splat` was not tested for this report.

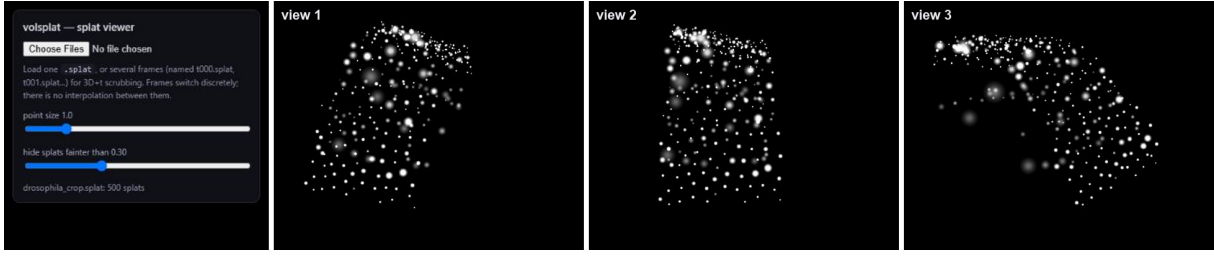


Figure 7: The browser viewer with the *Drosophila* fit of Section 6 (500 Gaussians, $64 \times 128 \times 128$ crop, z stretched $5\times$, Gaussians with display brightness below 0.3 hidden). Left: control panel. Right: three orbit angles of the same fit; the curved sheet is the embryo surface, the bright points its nuclei. Screenshots of the running viewer.

8 Time-resolved extension and playback

Three representations. Goal 3 of the brief compares per-frame Gaussian sets with shared identity against 4D Gaussians. We implemented three variants (`volsplat/temporal.py`). *Shared identity* fits one set per time point; the first is fitted from scratch and each later one starts from its predecessor, so that Gaussian i keeps describing the same structure. Storage grows with the number of frames, and time is discrete. *Native 4D* gives each Gaussian a time centre and a temporal extent (13 numbers instead of 11); its density is the product of a spatial and a temporal Gaussian, so a Gaussian fades in and out but does not move, and motion has to be represented by successive Gaussians. *Deformation* keeps one set whose positions follow a quadratic path in time (velocity and acceleration, 17 numbers), in the spirit of deformation-based 4D splatting [10]. Both continuous-time variants can be rendered at any t .

Data. All temporal results are on a synthetic 4D phantom: $16 \times 32 \times 32$ voxels and eight Gaussian blobs (widths 2–3 voxels) moving along smooth random paths, one of which divides. The motion is small: each blob stays within 1.5 voxels of its mean position, less than its own width, while the two daughters of the dividing blob move apart by 1.5 voxels per frame. We have no usable real-data comparison. The *Drosophila* time-series run in the repository used the crop from before the region-of-interest fix (Appendix B), so it fitted the empty interior of the embryo; a corrected three-frame run on the cluster, described in our progress report, left no outputs in the repository. Neither is reported here.

A first comparison, and its two flaws. Our first comparison gave every variant the same 300 Gaussians and found shared identity far ahead (39.3 against 25.9 and 22.8 dB mean PSNR over four frames). It had two flaws. Shared identity stored $3.4\times$ more numbers (13,200 against 3,900 and 5,100), and its “interpolation” score rendered the continuous-time models half-way between frames but compared the result with a neighbouring *training* frame, so it did not measure interpolation at all. We therefore repeated the comparison with matched storage and genuinely held-out frames.

Matched storage, held-out frames. We generated the phantom with seven frames, fitted every method on the four even frames only and scored it on the three odd frames it never saw (`scripts/temporal_matched.py`, three seeds). All methods stored about 13,200 numbers: four per-frame sets of 300 Gaussians, 1,015 native-4D Gaussians, or 776 deformation Gaussians. Per-frame sets cannot render between frames, so we scored them in two ways: the nearest earlier frame, which is what a frame slider shows, and an equal blend of the two neighbouring frames.

Representation	Gaussians	Numbers	Fit frames (dB)	Held-out frames (dB)
One set per frame, nearest frame cross-fade of two frames	4×300	13,200	37.5 ± 1.4	19.8 ± 1.8 22.7 ± 2.0
Native 4D	1,015	13,195	18.5 ± 2.4	16.3 ± 1.7
Deformation	776	13,192	20.5 ± 1.0	19.0 ± 0.6
Native 4D	300	3,900	23.6 ± 2.0	19.6 ± 0.4
Deformation	300	5,100	22.1 ± 1.3	19.7 ± 0.7

Table 11: Time-series representations on the synthetic phantom: mean PSNR on the four frames used for fitting and on the three held-out frames between them (mean $\pm$ standard deviation over three seeds). Upper block: matched storage; lower block: the continuous-time models with 300 Gaussians.

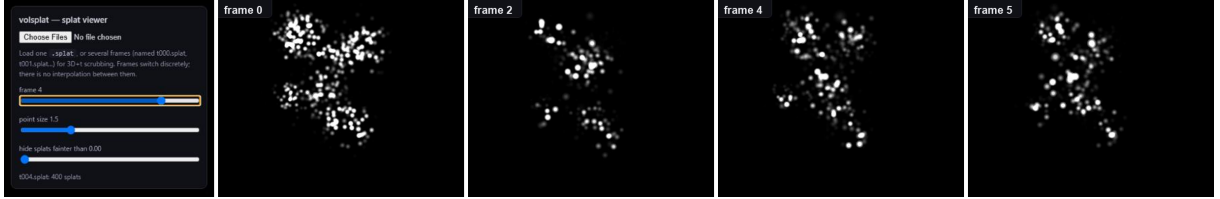


Figure 8: Playback of a synthetic time series (six frames, 400 Gaussians per frame, shared identity) in the browser viewer: control panel with the frame slider, and frames 0, 2, 4 and 5. Frame 0 was fitted from scratch and looks denser; later frames start from their predecessor. Screenshots of the running viewer.

Because more Gaussians made the continuous-time models fit *worse*, we also fitted them with 300 Gaussians as a control. Table 11 gives the results.

Per-frame sets reproduced their own frames far better than either continuous-time model (37.5 dB, against 18.5 to 23.6 dB). Between frames, the simplest option won: an equal blend of the two neighbouring per-frame fits reached 22.7 dB on the held-out frames, against about 20 dB for showing the nearest frame and 16.3–19.7 dB for the continuous-time models. At matched storage the continuous-time models fitted even their own frames worse than with 300 Gaussians (native 4D 18.5 against 23.6 dB), so their optimisation, not only their representation, limited them; we did not tune it. On smoothness, the per-frame Gaussians moved smoothly (mean acceleration 0.15 voxels per frame²), and scoring each frame with the previous frame’s model gave 16.2 dB, which mostly measures the motion between frames rather than drift. In the run without interruptions, per-frame sets took about 7 minutes on the laptop CPU, deformation 17 and native 4D 25. On this small phantom, then, the extra machinery of 4D Gaussians did not pay off; whether it would on real embryos, with larger motion and many divisions, remains open.

Playback. The viewer of Section 7 plays a time series as a sequence of per-frame `.splat` files with a frame slider (Figure 8). Frames switch discretely; nothing is interpolated, and the continuous time of the native-4D and deformation models is not exported, because `.splat` has no time axis. Before the fix mentioned in Section 7, the viewer centred each frame on its own centre of mass, which partly removed the motion it was meant to show.

Limitations. The phantom is small and synthetic, and real nuclei were never tracked with these representations. Native-4D Gaussians cannot move. All variants used learning rates from static fitting, including the position rate that Step 7 found to be too small. The cell division is handled only implicitly, and there is no standard file format for 4D Gaussians.

9 Comparison with the state of the art at matched file size

Our own fits improved on the naive setup, but they are slow (minutes per small crop on a CPU) and were never designed for whole recordings. If Gaussian blobs are to win anywhere, the strongest existing tool must win. We therefore compared Luxar [3] with standard codecs at identical file size on the *C. elegans* frames, scored by four detectors against human labels.

9.1 Setup

Compressors. Luxar with its default settings, 1,000 iterations and anisotropic voxel size, at budgets K from 500 to 6.4×10^4 splats; its size is the `.gsplats.zarr` store on disk, metadata included. Renders are mapped to raw intensities by a least-squares gain and offset (two numbers, not counted, which slightly favours Luxar). We also ran Luxar’s own calibration, a blind-spot held-out PSNR [12], to obtain its recommended budget K^* . JPEG2000 (OpenJPEG) codes each xy slice with a 2D wavelet, slices stored as components of one codestream, and is bisected to *exactly* the bytes of every Luxar fit. JPEG-XL codes each slice and is bisected to 50–800 KiB.

Detectors. (a) LoG: local maxima ranked by scale-normalised LoG response at scales $\sigma = 0.2D, 0.27D, 0.33D$ and $0.4D$, top N kept, N = number of labels. (b) Watershed: background-subtracted, Otsu-thresholded foreground split by a distance-transform watershed in physical units, parameters fixed on raw volumes. (c) Cellpose 3 *nuclei* model, 2D per slice stitched across z , and (d) the same model in its 3D mode. On the raw volumes they find 94%, 89%, 75% and 82% of the labelled nuclei. Detections match labels one-to-one within $0.6D$; Table 13 varies the radius.

Statistics. Each frame counts once: per-frame mean difference between Luxar and JPEG2000 at identical bytes, Wilcoxon signed-rank tests over the 12 frames, Holm-corrected over the eight detector–regime comparisons, and 95% cluster-bootstrap intervals over frames. Pass/fail rules were written down and committed before each new measurement (Appendix C); the two that failed are reported as failed.

A baseline pitfall. Passing a (z, y, x) array to a common JPEG2000 binding encodes a z – y image with one component per x column, so the wavelet never runs along x . That configuration, which we used at first, cost about 10 dB at $220 \times$ and made splats appear far better than JPEG2000 beyond $200 \times$. Fixing it reversed our first conclusion. A regression test now checks the codestream header.

9.2 Results

PSNR says JPEG2000; the detectors say it depends. JPEG2000 had higher PSNR on every frame in both regimes and in all 80 of 80 pairs, by 1.0–6.8 dB (Table 12). Below $300 \times$, 3D Cellpose favoured the splats (Luxar – JPEG2000 = +3.5 points of nuclei kept, 10 of 12 frames, $p_{\text{Holm}} = 0.010$), whereas the watershed and 2D Cellpose favoured JPEG2000 (–4.0 and –9.4 points, every frame, $p_{\text{Holm}} = 0.004$ and 0.004). The LoG detector showed no consistent difference (+1.3 points, 8 of 12 frames, $p_{\text{Holm}} = 0.110$); it favoured splats on embryo 2 (+2.4) but not on embryo 1 (+0.3). Beyond $300 \times$ (up to $603 \times$) every detector favoured JPEG2000 on 11–12 of 12 frames, by –16.6 (LoG) to –23.2 (2D Cellpose) points, while JPEG2000 still kept 98–103% (LoG) and 96–100%

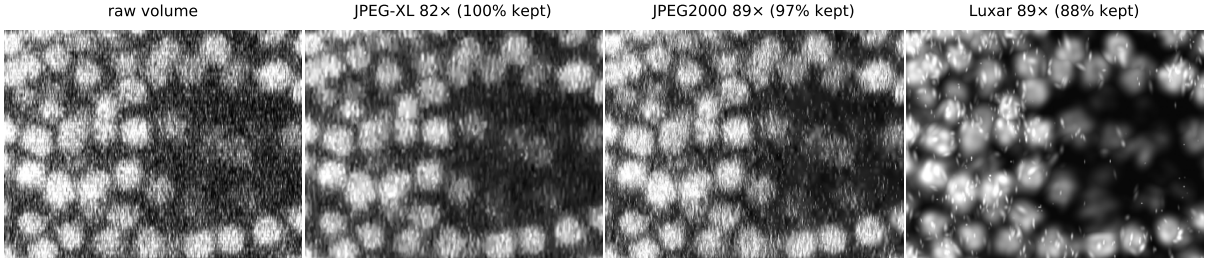


Figure 9: The same region of embryo 1, t194 (3-slice maximum projection) from the raw volume, JPEG-XL, and JPEG2000 and Luxar at identical bytes; titles give whole-frame nuclei kept by 2D Cellpose. Both codecs keep the texture of nuclei; Luxar renders smooth blobs on a black background with thin streaks from elongated splats.

Luxar – JPEG2000	below 300 ×		300 × and beyond	
	mean [95% CI]	L : J	mean [95% CI]	L : J
PSNR (dB)	−2.4 [−2.7, −2.1]	0 : 12	−1.4 [−1.6, −1.3]	0 : 12
LoG	+1.3 [+0.0, +2.7]	8 : 4	−16.6 [−23.0, −10.2]	1 : 11
Watershed	−4.0 [−5.3, −2.7]	0 : 12	−19.3 [−23.8, −15.0]	0 : 12
Cellpose (2D)	−9.4 [−11.4, −7.3]	0 : 12	−23.2 [−28.0, −18.3]	0 : 12
Cellpose (3D)	+3.5 [+1.8, +5.5]	10 : 2	−18.2 [−23.8, −12.8]	0 : 12

Table 12: Luxar minus JPEG2000 at identical bytes, frame level: mean over 12 frames of the per-frame mean difference (nuclei kept in percentage points; PSNR in dB), 95% cluster-bootstrap interval over frames, and frames favouring Luxar : JPEG2000 (L : J). Bold: Holm-corrected Wilcoxon $p < 0.05$.

(watershed) of nuclei. Every individual pair is shown in Figure 10. JPEG-XL was nearly lossless up to its maximum distance (82–120×; worst case 91%).

Why the detector matters. Smooth splats can act as a denoiser for detectors that pool over a volume: under LoG and 3D Cellpose, nuclei kept often exceeded 100% (Luxar at 45 × to 94 ×: 98–106% and 104–118%). Per-slice and threshold-based segmenters instead see smooth blobs on a black background with streaks (Figure 9). The same Cellpose network shows this clearly. At 45 × to 94 ×, JPEG-XL minus splats was +4 to +15 points of nuclei kept in 2D mode but −18 to −4 in 3D mode, where the 5-point penalty we had tested for in advance appeared on 0 of 12 frames. The same weights behave differently, so the penalty is not a fixed property of

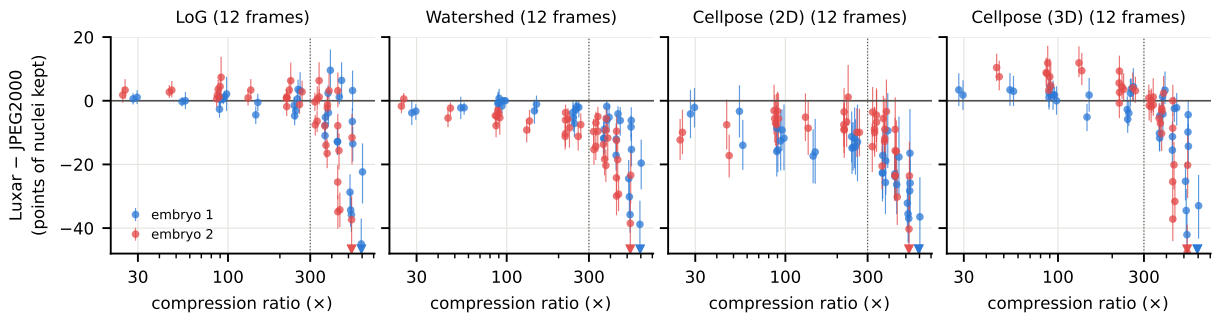


Figure 10: Every Luxar fit against JPEG2000 at identical bytes: difference in nuclei kept (points; bars: 95% bootstrap intervals over nuclei) for the four detectors. Above zero, splats keep more nuclei; triangles: below −48 points. Dotted line: 300 ×.

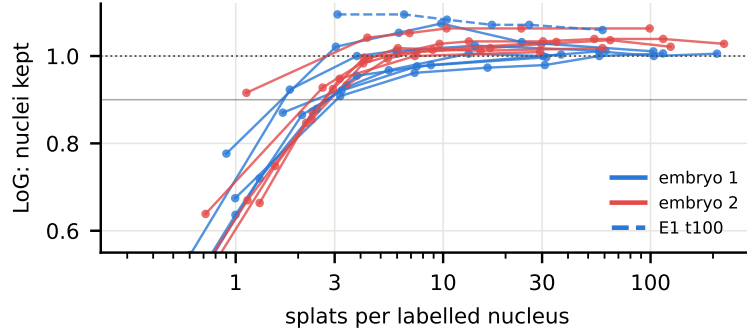


Figure 11: The budget cliff: LoG nuclei kept against splats per labelled nucleus, every Luxar fit on all frames (dashed: embryo 1, t100).

Radius	LoG			Cellpose (2D)		
	raw	JXL	Luxar	raw	JXL	Luxar
0.4D	1009	1.00	1.02	658	1.01	0.84
0.6D	1032	1.00	1.01	788	0.97	0.88
0.8D	1034	1.00	1.01	796	0.99	0.90

Table 13: Match-radius sensitivity (four first frames): nuclei found in the raw volumes and nuclei kept near $90\times$ (mean).

the learned model; this supports an explanation involving processing geometry (slices versus the whole volume). The 3D mode also combines and post-processes predictions differently, so switching modes establishes the difference in behaviour but does not isolate its exact cause. The watershed sits in between (+0 to +5 points; under the pre-set 5 points on 11 of 12 frames).

The budget cliff. Splats lose nuclei abruptly once they become scarce (Figure 11): under LoG, nuclei kept fell below 90% at 1.1–3.1 splats per labelled nucleus on all 12 frames, as pre-registered (≤ 3.5). Over budgets $K \geq 1000$, Luxar’s PSNR varied by at most 2.6 dB per frame. Our pre-registered test that PSNR misses the cliff within $K = 1,000$ –16,000 held on only 4 of 8 added frames: on smaller frames the cliff lies below that range. Survival, the share of the raw volume’s 100 strongest LoG detections still found, needs no labels and tracks nuclei kept across fits (Spearman $\rho = 0.66$, $n = 80$), so it can flag the cliff.

Luxar at its own budget. Luxar’s calibration recommended $K^* = 64,000$ on all 4 calibrated frames of both embryos, the top of our grid, with held-out PSNR still rising. There, at 25–30 $\times$, it kept 101–103% (LoG), 96–100% (watershed), 89–98% (2D Cellpose) and 102–105% (3D Cellpose) of nuclei; JPEG2000 at the same bytes kept 99–102%, in seconds rather than 17–46 min of fitting per volume on an A100 GPU (5–40 min on an H100).

Robustness. Three independent Luxar fits of embryo 1 t194 at each of three budgets differed by at most 2.1 (LoG), 3.5 (watershed) and 3.7 (2D Cellpose) points. The conclusions do not depend on the match radius (Table 13). On embryo 1 t100, with larger early nuclei, LoG favoured splats more strongly (8–11 points at 159–537 $\times$).

10 Discussion

What was solved, and by how much. On the fitting side, the changes of Section 5 turned a fit that merely looks right into one that keeps most nuclei: 52% $\rightarrow$ 71% of nuclei found by seeding one blob per nucleus, 13 $\rightarrow$ 19 of 31 faint nuclei with a size cap, and the diagnosis that the position optimiser was under-scaled rather than stuck. Against human labels, however, the fitted representation ranked nuclei better than raw brightness but, in the top 100, worse than a classical blob filter (level in the top 200), and every other use we tested lost to a simple baseline. On the storage side, we showed with 12 frames and four detectors that at the same bytes, state-of-the-art Gaussian compression is neither safer nor less safe than JPEG2000 in general: 3D Cellpose kept +3.5 points more nuclei in splats below 300 $\times$, 2D Cellpose -9.4 points fewer, and every detector lost 17–23 points with splats beyond 300 $\times$. PSNR ranked JPEG2000 first on every frame and could not reveal any of this. It is not wrong, but insufficient: object-level, size-matched, detector-specific evaluation should accompany Gaussian volume compression.

Limitations. The fitting experiments used detector-derived references on *Tribolium*, and several of them a single seed; the human-label check rests on 29 nuclei in one crop. The state-of-the-art comparison covers one organism (two embryos, 12 frames, with frames rather than embryos as the units of replication), one Gaussian compressor at default settings (tuned or entropy-coded splats may fare better), and a slice-wise JPEG2000 (a 3D wavelet would likely be stronger still). The fixed nucleus diameter D suits early, larger nuclei less well. Our fitting model ignores the microscope’s blur and photon noise and treats background as a sum of broad blobs; several findings (background capacity, widening) are symptoms of that. Much of the effort went into finding and fixing our own evaluation bugs (Appendix B); one of them, the JPEG2000 axis error, had reversed our first conclusion.

Possible next steps. In informal feedback at the chair it was pointed out to us that biologists store raw data losslessly and that lossy image codecs such as JPEG are rarely used in practice. Our results agree that viewing and measuring need separate validation: splats can look convincing and still lose nuclei for one detector while gaining them for another. We therefore see compression that biologists would accept as the natural continuation; this is our proposal, not an agreed plan. On one *C. elegans* frame (embryo 1, t150; 8-bit, 12.7 MB) standard lossless coders reached 1.9–4.2 $\times$: JPEG-XL lossless per slice 4.2 $\times$, JPEG-LS 2.9 $\times$, JPEG2000 lossless 2.7 $\times$, Zstandard 2.5 $\times$. Three directions follow: (i) noise-bounded (near-lossless) coding, where every voxel may change by less than the camera noise [13], scored with the object-level protocol of Section 9; (ii) video codecs, which exploit the similarity of neighbouring slices and time points; (iii) representations that adapt to local image content, such as the adaptive particle representation [14], stored in chunked, compressed array formats such as Zarr. Success would be judged by whether cells can still be detected and tracked, using the Cell Tracking Challenge measures [1], rather than by PSNR.

11 Conclusion

Gaussian blobs can represent light-sheet volumes compactly, and careful fitting (stretched blobs, one start per nucleus, a size cap) makes them keep most nuclei. Measured against human labels and at identical bytes, however, they do not beat simple alternatives: a blob filter ranks nuclei at least as well, and against JPEG2000 the state-of-the-art Gaussian compressor gains or loses nuclei depending on the detector, even between the 2D and 3D modes of one network, and

loses them abruptly below about three splats per nucleus, a budget that PSNR cannot identify. The reusable outcome of the project is the evaluation itself: compare at matched bytes against a verified codec, score with the detector that will be used, and fix the pass/fail rules before looking at new data.

References

- [1] Martin Maška et al. “The Cell Tracking Challenge: 10 years of objective benchmarking”. In: *Nature Methods* 20 (2023), pp. 1010–1020.
- [2] Bernhard Kerbl et al. “3D Gaussian Splatting for Real-Time Radiance Field Rendering”. In: *ACM Transactions on Graphics* 42.4 (2023).
- [3] Loïc A. Royer. *Luxar: Gaussian splatting for microscopy and scalable interactive web visualisation of multidimensional scientific data*. Zenodo, <https://zenodo.org/records/22912050>. Preprint; software at <https://github.com/royerlab/luxar>. 2026.
- [4] Di Kong et al. *GaussianPile: A Unified Sparse Gaussian Splatting Framework for Slice-based Volumetric Reconstruction*. arXiv:2603.20611. 2026.
- [5] Xiaoduo Li and Quan Gu. *GSToken: Geometry-Structured Gaussian Tokens for Compact 3D Medical Image Representation*. arXiv:2608.17425. 2026.
- [6] Michael R. Martin et al. *Fixed-Budget Gaussian Volume Encoding with Structure-Aware Allocation*. arXiv:2608.14112. 2026.
- [7] Ruyi Zha et al. “R²-Gaussian: Rectifying Radiative Gaussian Splatting for Tomographic Reconstruction”. In: *Advances in Neural Information Processing Systems*. 2024.
- [8] Yingtai Li et al. “3DGR-CT: Sparse-view CT reconstruction with a 3D Gaussian representation”. In: *Medical Image Analysis* 103 (2025), p. 103585.
- [9] Fernando Amat et al. “Fast, accurate reconstruction of cell lineages from large-scale fluorescence microscopy data”. In: *Nature Methods* 11.9 (2014), pp. 951–958.
- [10] Guanjun Wu et al. “4D Gaussian Splatting for Real-Time Dynamic Scene Rendering”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024. DOI: 10.1109/CVPR52733.2024.01920.
- [11] Maik Dannecker et al. “Fast and Explicit: Slice-to-Volume Reconstruction via 3D Gaussian Primitives with Analytic Point Spread Function Modeling”. In: *Medical Imaging with Deep Learning (MIDL)*. 2026.
- [12] Joshua Batson and Loïc Royer. “Noise2Self: Blind Denoising by Self-Supervision”. In: *Proceedings of the 36th International Conference on Machine Learning*. 2019, pp. 524–533.
- [13] Balázs Balázs et al. “A real-time compression library for microscopy images”. In: *bioRxiv* (2017). doi:10.1101/164624.
- [14] Bevan L. Cheeseman et al. “Adaptive particle representation of fluorescence microscopy images”. In: *Nature Communications* 9 (2018), p. 5160.
- [15] David S. Taubman and Michael W. Marcellin. *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Kluwer Academic Publishers, 2002.
- [16] Jyrki Alakuijala et al. “JPEG XL next-generation image compression architecture and coding tools”. In: *Applications of Digital Image Processing XLII, Proc. SPIE* 11137. 2019.
- [17] David A. Koff and Harry Shulman. “An overview of digital compression of medical images: can we use lossy image compression in radiology?” In: *Canadian Association of Radiologists Journal* 57.4 (2006), pp. 211–217.

References

- [18] Vladimír Ulman et al. “An objective comparison of cell-tracking algorithms”. In: *Nature Methods* 14 (2017), pp. 1141–1152.
- [19] Tony Lindeberg. “Feature detection with automatic scale selection”. In: *International Journal of Computer Vision* 30.2 (1998), pp. 79–116.
- [20] Carsen Stringer et al. “Cellpose: a generalist algorithm for cellular segmentation”. In: *Nature Methods* 18 (2021), pp. 100–106.
- [21] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015.

A Glossary

3D Gaussian blob (splat) A soft, possibly stretched and rotated ball of brightness, brightest in the middle and fading outwards; “Gaussian” describes how it fades.

Voxel A 3D pixel: one brightness value at one point of the volume.

Light-sheet microscope A microscope that illuminates the sample with a thin sheet of light and images it layer by layer, giving fast, gentle 3D recordings of living embryos.

Frame One 3D snapshot at one time point; a recording is a series of frames.

Maximum projection A 2D picture of a volume that keeps, along each line of sight, the brightest voxel.

Ground truth / human labels Nucleus positions marked by experts; checks against them are not circular.

Detector-derived reference Nucleus positions found by a program; a method can look good simply by agreeing with that program.

Phantom Synthetic test data with a known answer.

PSNR Peak signal-to-noise ratio, in dB: how close a reconstruction is to the data; higher is better and +3 dB roughly halves the squared error. It says nothing about whether individual objects survive.

Alpha-compositing The rule in 3D Gaussian splatting that each blob partly hides the blobs behind it; correct for opaque surfaces, wrong for transparent, glowing volumes.

Voxel query Evaluating the blob sum at every voxel centre and comparing it with the data, without a camera.

Budget (K) The number of blobs (splats) allowed.

Initialisation / seeding Where blobs are placed before fitting starts.

Learning rate How far each fitting step moves the parameters.

Migration A blob’s nearest nucleus changes between start and end of fitting.

Densification Adding blobs during fitting by splitting or cloning existing ones where the error is large.

Size cap An upper limit on the scale of the blobs that started on nuclei.

Recall, precision, F1 Share of reference nuclei found; share of detections that are real; their harmonic mean.

Top- N comparison Comparing methods at the same number of detections, because recall grows with the number of detections.

Nuclei kept Labelled nuclei found after compression divided by those found in the raw volume by the same detector; 1.0 means the same number was found, not necessarily the same individual nuclei.

Points Percentage points of nuclei kept, *e.g.* 0.90 against 0.86 is 4 points.

Compression ratio Original size divided by compressed size.

Matched bytes Two compressed files of exactly the same size on disk.

Codec A compression method (coder–decoder), *e.g.* JPEG2000 or JPEG-XL.

Lossless / near-lossless Decompressed data identical to the original / small, bounded changes allowed. *Noise-bounded* coding is one near-lossless scheme, in which every voxel changes by less than the camera noise.

LoG Laplacian of Gaussian, a classical blob detector that responds to bright round spots of a given size.

Watershed A classical segmentation that separates touching bright regions like water filling valleys.

Cellpose 2D / 3D A widely used deep-learning segmenter; the 2D mode works slice by slice and joins the slices, the 3D mode combines predictions from three viewing directions.

B Evaluation and pipeline bugs found

Wilcoxon signed-rank test, Holm correction A test of whether paired differences are centred on zero; a correction that keeps the error rate under control across several tests.

Budget cliff The sudden loss of nuclei once fewer than about three splats per nucleus remain.

Survival Share of the raw volume’s 100 strongest LoG detections still found after compression; needs no labels.

Pre-registration Writing down pass/fail rules before the data they apply to are seen.

`.ply` / `.splat` File formats for Gaussian splats: the 3D Gaussian splatting PLY layout (68 bytes per Gaussian) and the compact antimatter15 layout (32 bytes).

Point sprite A flat, round dot that always faces the camera; our viewer draws each Gaussian as one.

Shared identity One set of Gaussians per time point, each fitted starting from the previous one, so that Gaussian i keeps describing the same structure.

Native 4D Gaussian A Gaussian with an extent in time as well as space; it fades in and out around its time centre but does not move.

Deformation model One set of Gaussians whose positions move along a low-order polynomial in time.

Held-out frame A time point not used for fitting, on which a method is scored.

B Evaluation and pipeline bugs found

Bug	Effect	Status
Crop centred on the bounding box	Hollow embryo: crops fitted empty interior, not nuclei	Fixed; crop anchored at the pole
Cubic detection window in voxels	Nuclei merged along z ; 5/29 instead of 22/29 recoverable	Fixed; physical window
Matching radius in voxels	Distant z -pairs accepted as matches	Fixed; matched in μm
Position learning rate not rescaled	Blobs moved < 1 voxel; mistaken for a physical barrier	Diagnosed (Step 7)
Assignment followed by radius filter	Valid matches dropped	Fixed; saved results rescored
Flat regions reported as peaks	Every plateau voxel a “nucleus”	Fixed (prominence guard)
Rankings compared at unequal budgets	A ranking judged better at a small budget lost at a large one	Fixed; matched budgets
JPEG2000 axis error	Wavelet never ran along x ; ≈ 10 dB lost; first conclusion reversed	Fixed; header test
Watershed scorer used unconfigured voxel size	Too tight effective radius	Fixed; test added
Interrupted Cellpose cache (all zeros)	One volume scored as 0% kept; 2D deficit -13.0 instead of -9.4	Fixed; atomic writes, cache check

Table 14: Bugs that changed a number or a conclusion before they were caught. Each evaluation bug is now pinned by one of the 24 regression tests in `scripts/test_metrics.py`.

C Pre-registered decision rules

Before each new batch of the state-of-the-art comparison was analysed, the pass/fail rules were committed to the repository (`paper/DECISION_RULES.md`); claims follow the verdicts and were not adjusted afterwards. Table 15 lists the main rules and outcomes.

Rule	Criterion (fixed in advance)	Outcome
Cliff (8 added frames)	LoG keeps $\geq 90\%$ only above ≤ 3.5 splats per nucleus, on ≥ 6 of 8 frames	Holds on 8 of 8
PSNR misses the cliff	Luxar PSNR varies < 3 dB over $K = 1,000$ – $16,000$ while LoG nuclei kept varies ≥ 0.15 , on ≥ 6 of 8	Fails: 4 of 8; reported narrowed
Detector dependence (2D Cellpose)	JPEG-XL keeps ≥ 5 points more than Luxar at $K = 16,000$, on ≥ 6 of 8	Holds on 8 of 8
Stitching (3D Cellpose)	Same gap ≥ 5 points on $\geq 75\%$ of frames would rule out slice-wise processing as the cause	Gap on 0 of 12: same model behaves differently in 3D mode
Learned vs classical (watershed)	Gap < 5 points on $\geq 75\%$ of frames	11 of 12
Luxar’s own budget K^*	Keeps $\geq 95\%$ of nuclei under LoG	Holds on all calibrated frames
Splats vs JPEG2000 at matched bytes	Splats may be called better only where the paired interval lies above zero	Reported per detector and regime (Table 12)

Table 15: Main pre-registered rules of the state-of-the-art comparison and their outcomes. Two earlier claims were withdrawn on these grounds: that splats beat JPEG2000 beyond $200 \times$ (an artefact of the JPEG2000 bug) and that the 2D Cellpose penalty reflects a learned model (refuted by its 3D mode).

D Reproducibility

Code, rules and results are in the project repository,
<https://github.com/asadrizvi64/DeepLearning-Seminar-TeamProject>.

- Regression suite: `python scripts/test_metrics.py` (24 tests).
- State-of-the-art comparison: `bash scripts/fidelity/reproduce.sh` regenerates every number, table and figure of Section 9 from the per-frame result files and asserts the worded claims; `paper/CLAIMS.md` maps each claim to its evidence. In this report, make sync copies the generated numbers and tables.
- Human-label experiment: `scripts/rerank_factorial.py`. Size cap: `runs/scale_cap_rescore/`. Akim’s archive, summarised: `runs/colleague_all_runs.csv` (the raw archive is kept outside the public repository).
- Luxar fits ran on the TU Dresden ZIH clusters (A100 and H100 GPUs); codecs and detectors on a laptop CPU, Cellpose 3D on the cluster.
- Earlier fitting results carry `git_dirty=True`: the recorded commit alone does not reconstruct their code. Two analyses could not be rescored after the matcher fix because their outputs were not saved: the loss-weighting experiment (not used here) and the end-of-fit width comparison behind Step 8 (`scripts/stage3_trajectory.py`), which is reported with that caveat. Where rescoring was possible, the fix moved counts by at most one nucleus.